

Ejercicio árboles

1. Explica brevemente los siguientes conceptos:

- a. Nodo
- b. Raíz
- c. Hijo
- d. Hoja
- e. Nivel y altura
- f. Subárbol

Nodo es una estructura base del árbol.
Almacena un valor y tiene conexiones (punteros) a otros nodos.

En un árbol binario, se tiene como máximo dos conexiones: un hijo izquierdo y un hijo derecho.

Raíz Es el único nodo que no tiene un "padre"; es el ancestro original del cual descienden todos los demás nodos.

Hijo Es un nodo que desciende directamente de otro.
Al nodo del que desciende se le llama padre.

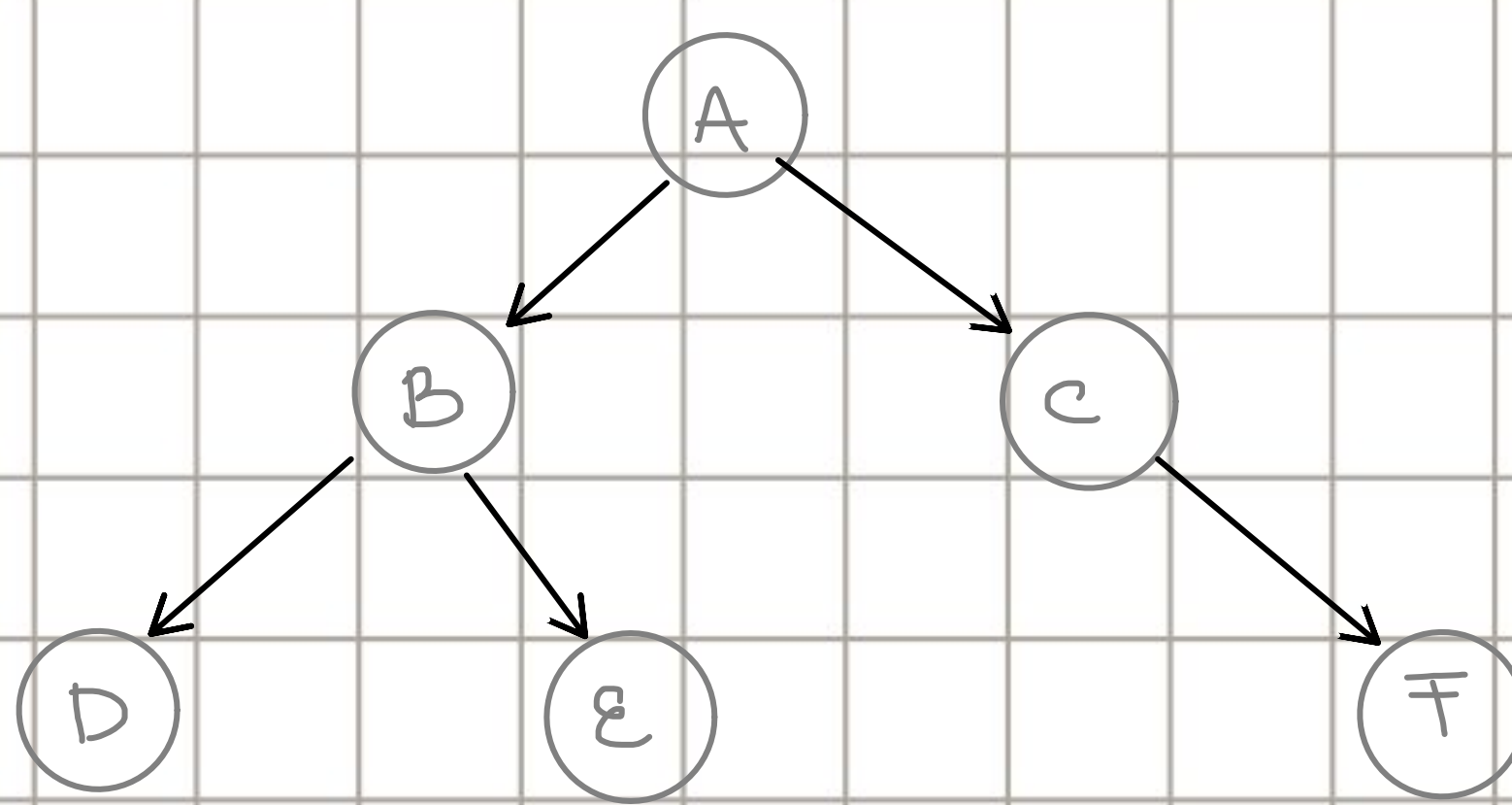
Hoja Es un nodo que no tiene hijos.
Es un punto final en una rama del árbol.

Nivel Distancia o número de enlaces desde la raíz.

Altura Distancia máxima desde la raíz hasta la hoja más lejana.

Subárbol Es un árbol formado por un nodo (raíz) y todos sus descendientes.

2. Dibuja un árbol binario que cumpla con:
- a. Raíz con valor A.
 - b. Dos hijos: B y C.
 - c. B tiene hijos D y E.
 - d. C tiene solo un hijo derecho F.



3. Implementa una clase `NodoBinario<T>` y `ArbolBinario<T>` que permita crear un árbol y recorrerlo en preorden.

```
public class NodoBinario<T> { 8 usages

    private T valor; 3 usages
    private NodoBinario<T> izquierdo; 3 usages
    private NodoBinario<T> derecho; 3 usages

    public NodoBinario(T valor) { no usages
        this.valor = valor;
        this.izquierdo = null;
        this.derecho = null;
    }

    public NodoBinario getIzquierdo() { 1 usage
        return izquierdo;
    }

    public void setIzquierdo(NodoBinario<T> izquierdo) { no usages
        this.izquierdo = izquierdo;
    }

    public NodoBinario<T> getDerecho() { 1 usage
        return derecho;
    }

    public void setDerecho(NodoBinario<T> derecho) { no usages
        this.derecho = derecho;
    }

    public T getValor() { 1 usage
        return valor;
    }

    public void setValor(T valor) { no usages
        this.valor = valor;
    }
}
```

```
public class ArbolBinario<T> { no usages

    private NodoBinario<T> raiz; 1 usage

    public ArbolBinario() { no usages
        this.raiz = null;
    }

    public void recorridoPreOrdenRec(NodoBinario nodo) { 2 usages
        if(nodo != null) {
            System.out.println(nodo.getValor());
            recorridoPreOrdenRec(nodo.getIzquierdo());
            recorridoPreOrdenRec(nodo.getDerecho());
        }
    }
}
```

4. Agregá métodos al árbol que permitan:
- Contar cuántos nodos tiene el árbol.
 - Contar cuántas hojas tiene.
 - Agregá un método `buscar(int valor)` que devuelva `true` si el valor está en el árbol.

```
public int contarNodos() { no usages
    return contarNodosAux(this.raiz);
}

private int contarNodosAux(NodoBinario nodo) { 3 usages
    int res = 0;
    if(nodo != null) res = contarNodosAux(nodo.getIzquierdo()) + 1 + contarNodosAux(nodo.getDerecho());
    return res;
}
```

```
public int contarHojas() { no usages
    return contarHojasAux(this.raiz);
}

private int contarHojasAux(NodoBinario<T> nodo) { 3 usages
    if (nodo == null) return 0;

    if (nodo.getIzquierdo() == null && nodo.getDerecho() == null) return 1;

    return contarHojasAux(nodo.getIzquierdo()) + contarHojasAux(nodo.getDerecho());
}
}
```